GITHUB CLONE + SAP CONNECTION

Connect your own app clone to SAP

A practical guide to cloning the Briefing Agent and connecting the running app to an authorized SAP S/4HANA system.

STEP 1	STEP 2	STEP 3	STEP 4
GitHub code	Clone & run	Configure access	Brief

Important distinction
This repository is a separate web application, not an SAP add-on installed inside the SAP server. Run the clone as its own app; it connects to your SAP tenant through APIs approved by your SAP administrator.

Repository

https://github.com/sham-cyber/Brief

Each clone or deployment is a separate app instance. It needs its own database and sign-in configuration. SAP access is not copied from the original project.

What the app brings together

SAP internal records
Approved SAP business APIs, mapped data areas, and permitted company or plant scope.

Public research
Tavily searches public sources separately; its key and provider billing are distinct from SAP.

Separate evidence
Public citations and SAP records appear in different sections of the brief.

AI status
AI is off in this app. Results are source-based, without AI-written claims.

Approval required. An SAP administrator must authorize the OAuth client, API, fields, and organizational scope for the clone.

1. Clone the repository

Install Git, Node.js 24, pnpm 10, and PostgreSQL 16 or a compatible PostgreSQL server. Then clone and install the locked dependencies:

```
git clone https://github.com/sham-cyber/Brief.git
cd Brief
pnpm install --frozen-lockfile
```

2. Configure server secrets

Use Replit Secrets or your hosting provider's secret manager. Set values there—never put them in source files or GitHub.

Sign-in / owner

`CLERK_PUBLISHABLE_KEY`
`CLERK_SECRET_KEY`
`VITE_CLERK_PUBLISHABLE_KEY`
`BRIEFING_OWNER_EMAIL`
`BRIEFING_START_PIN`

Database / encryption

`DATABASE_URL`
`USER_CREDENTIALS_ENCRYPTION_KEY`
Needed for database access and encrypted stored credentials.

Public search

`TAVILY_API_KEY`
Optional project key. It is separate from SAP OAuth credentials.

Keep private

Never put secret values, owner email, or PIN in source, commits, chat, screenshots, or this guide.

3. Initialize and start

For a fresh development database, set `DATABASE_URL` to that database, then apply the schema:

```
pnpm --filter @workspace/db run push
```

Recommended: import/open the repository in Replit, configure Secrets, and press **Run**. For manual development, start the API and web services in separate terminals:

```
PORT=8080 pnpm --filter @workspace/api-server run dev
PORT=21375 BASE_PATH=/ pnpm --filter @workspace/briefing-agent run dev
```

Local routing: outside Replit, configure a same-origin `/api` reverse proxy to the API service. The owner signs in with the Clerk account matching the configured owner email and enters the server-side PIN to start a brief.

See the repository README for the complete setup and hosting notes. Run database schema commands against a development database only.

2. Prepare the SAP connection

Ask the SAP administrator to issue a dedicated OAuth client for this app and confirm the API details for the target system.

Request these details

1. Approved SAP S/4HANA OData service root and OAuth token URL.
2. OAuth 2.0 client ID and secret with least-privilege permissions.
3. SAP Fiori base URL for source links, if available.
4. Allowed company codes or plant values and approved business areas.
5. For each business area: released entity set, date field, record ID, title field, permitted summary fields, and filter format.

Areas available for mapping

Operations
Store · Production · Maintenance · Inventory

Commercial
Sales · Purchase · Order · Delivery

Other
HR · Power / Electrical / Solar · Scrap

Mapping rule
Only map areas and fields that the SAP administrator approves for the app.

Do not guess endpoint paths or fields. They depend on the SAP product, release, activated APIs, and organization. The app currently uses OAuth client credentials and OData; API-key authentication is not enabled by default. Direct HANA SQL/JDBC/ODBC access is not enabled by this setup.

Connection and data flow

OAuth + OData

The app server obtains an access token and calls the approved SAP API. Configure an owner-scoped connection in the SAP settings form. The test checks authentication and API metadata; it does not fetch operational records.

Separate Tavily research

SAP mode also requires an active Tavily key for public sources. SAP connection credentials and Tavily credentials are stored and billed separately; their results stay in separate brief sections.

Nothing crawls continuously. The app requests selected SAP and public data only when the user completes the form and presses **Brief**.

3. Configure and use SAP

1. Sign in as the configured app owner.
2. In **Source**, select **SAP S/4HANA**. Click the round **i** beside SAP for this guide, or the gear to open SAP settings.
3. Enter the OData service URL, OAuth token URL, Fiori base URL, optional scope, and approved company/plant values.
4. Enable approved data areas and map each area to the entity set and fields supplied by the SAP administrator.
5. Enter the OAuth client ID and secret in the protected settings form. Save, then click **Test connection**.
6. Confirm a separate Tavily key is active. Select the company, meeting time, briefing topics, and mapped SAP areas; press **Brief** and enter the PIN when prompted.

Ready indicators and troubleshooting

- ✓ **SAP + Tavily ready** means a saved connection, at least one mapped area, and an active Tavily key are available.
- SAP records and public research appear as separate evidence sections. Check that the company/plant scope is correct and follow the source links.
- If SAP or Tavily setup is incomplete, the app switches back to Public and shows an alert. No SAP records are requested until **Brief** is pressed.
- If an area is unavailable, confirm that it is mapped and enabled for the connection by the SAP administrator.

Security and disconnect

SAP OAuth credentials are encrypted on the app server; they are not saved in browser storage or GitHub. The server must have `USER_CREDENTIALS_ENCRYPTION_KEY` configured before saving credentials.

Removing the saved app copy does not revoke the OAuth client. Ask the SAP administrator to revoke the client in SAP too. Keep SAP credentials and the Tavily key separate.

References

Repository guide: GitHub README · Connect a clone to SAP
SAP APIs: SAP S/4HANA APIs · SAP OData APIs for Sales